Graphic Era deemed to be University DEHRADUN

PROJECT-BASED LEARNING

PHASE-II EVALUATION

ALGORITHMIC MAZE SIMULATOR

Team ID: DSCPP-III-2026-2338

Department of Computer Science & Engineering

Graphic Era (Deemed to be University), Dehradun

Academic Session 2026–27

Team & Mentor Details

Team Information

Team: HUSTLERS
Team ID: DSCPP-III-2026-2338
Semester: 3rd
Section: J, K
Domain: Maze path-finding

Team Members

1. Jaydeep Sharma – 2027835 (Lead)
2. Jaskirat Singh – 2027834

Mentor

Darshan Singh **Interactions:** 4

Jaydeep: C++ maze structure, generator and solvers. Jaskirat: manual controls and browser interface.

Phase-I Recap & Feedback Incorporated

Phase-I Status

Proposed a maze generator and solver using data structures and OOP. The Phase-II report describes a working C++ implementation with a browser interface.

Feedback Received

- Specific Phase-I feedback not reported.
- No mentor suggestions documented.
- No formal feedback record supplied.

Current Design Improvements

- Separate generator, solvers and game classes.
- DFS and BFS solving modes.
- Keyboard play and live web statistics.

These are reported Phase-II features; the report does not attribute them to specific Phase-I feedback.

Problem Statement & Finalized Objectives

Problem Statement

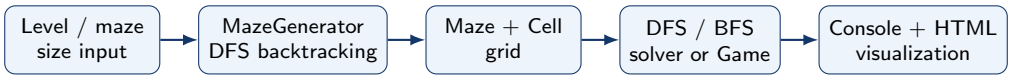
Creating and solving mazes manually takes time. The project generates mazes and demonstrates how search algorithms find a route from the start to the goal.

Finalized Objectives

1. Generate random mazes using DFS and backtracking.
2. Solve mazes using DFS (stack) and BFS (queue).
3. Support manual play with W/A/S/D and arrow keys.
4. Visualize visited cells, final paths and live statistics.

Current scope: C++ maze generation and solving, plus a browser interface. Reported completion: about 70%.

Updated System Architecture / Workflow



Major Components

- Maze, Cell, MazeGenerator
- Solver, DFSSolver, BFSSolver
- Game, Visualizer, HtmlExporter

Current Design

- Class-based C++ architecture
- Shared types in Common.h
- Browser controls and live stats

No database is used. The maze grid is passed to the solvers/game; results are displayed in the console or exported to index.html.

Module Design & Functional Requirements

Major Modules

1. Maze generation (DFS/backtracking)
2. DFS and BFS solvers
3. Manual play / Game
4. Visualizer and HTML export

Functional Requirements

- Generate mazes at chosen sizes.
- Find a start-to-goal path.
- Move with WASD / arrow keys.
- Show visited cells, path and stats.

Module Dependency

MazeGenerator builds the Maze grid. Solvers use the grid and parent pointers to reconstruct a path. Game handles manual moves; Visualizer and HtmlExporter display results.

The same maze data supports algorithm solving and manual play.

Technology Stack & Development Methodology

Technology Stack

- Programming: C++14
- Database: Not used
- Web: HTML, CSS, JavaScript
- Build: g++, build.bat
- Development Tool: VS Code, Live Server
- Version Control: Git / GitHub

Development Methodology

1. Design maze and solver classes.
2. Implement DFS maze generation.
3. Add DFS/BFS solving.
4. Add manual play and browser UI.
5. Build and test core functions.

C++ standard containers implement DSA; HTML/CSS/JavaScript make the results accessible and interactive.

Implementation of PBL Course Concepts

Course	Concepts Actually Implemented	Project Module
Data Structures in C	Grid/cell storage, stack for DFS, queue for BFS, parent pointers	Maze and DFS/BFS solvers
OOPs with C++	Classes, objects, encapsulation, abstraction, Solver base class and derived solvers	Maze, Solver, Game, Visualizer
Operating Systems	No OS-specific concept identified in report	Not reported
DBMS	No database implemented	Not used

Important: Only concepts actually implemented in the project are listed.

DFS uses a stack; BFS uses a queue and finds a shortest path in the unweighted maze.

Development & Implementation Progress

Module	Status	Work Completed
Maze generation	Completed	DFS recursive backtracking; maze levels/sizes
DFS + BFS solvers	Completed	Stack/queue search and path reconstruction
Manual play + web UI	Working / refinement	Keyboard movement, colour-coded cells and stats
Custom editor + extra algorithms	Pending	Maze editor, A*, Dijkstra, Greedy BFS, leaderboard

Overall Project Completion: 70% (reported)

Goal detection needs fixes; the custom editor and additional algorithms are planned rather than completed.

Working Prototype / Demonstration

Implemented Features

- Random maze generation
- DFS and BFS path solving
- WASD / arrow-key manual play
- Browser UI with speed slider and stats

Prototype / Output

Working prototype described in report.

Open `index.html` with Live Server.

Insert actual project screenshot here.

The report describes a working version but does not include a usable prototype screenshot for this slide.

Testing, Results & Validation

Test / Module	Expected Result	Observed Result	Status
Compilation	Build executable	`maze_game.exe` created	Pass
Maze generation	Generate supported sizes	11x11, 15x15, 21x21	Pass
BFS shortest path	Find shortest route	Level 1 solved in 25 steps	Pass

Results / Validation

DFS finds a valid path but not always the shortest. Manual movement passed testing; some goal-detection errors are still being fixed.

Results are taken from the report's testing table; no additional performance metrics are claimed.

GitHub Repository & Development Activity

Repository Information

- GitHub: Algorithmic Maze Simulator
- Branch: CS
- README: Not reported
- Contributors: Not confirmed
- Status: Codebase reported

Development Activity

- Initial Maze/Cell/Solver structure
- DFS maze generator added
- BFS and DFS solvers added
- Manual play and live stats added
- Individual commit counts not reported

Requirement: All team members should contribute and commit from their own GitHub accounts.

Repository: github.com/sen339853-rgb/ALGORITHMIC-MAZE-SIMULATOR-.git

Individual Contribution

Member	Role	Technical Contribution	GitHub Commits
Jaydeep Sharma	Team Lead	C++ structure, Maze/Cell classes, DFS generator, DFS/BFS solvers	Not reported
Jaskirat Singh	Team Member	Manual play, HTML/CSS interface, colour-coded cells, speed slider and stats	Not reported

Individual Evaluation

Each member should demonstrate the modules attributed to them and explain their implementation.

Only two team members and their responsibilities are identified in the supplied report.

Challenges, Remaining Work & Phase-III Roadmap

Challenges Faced

- Smooth and correct player movement
- Reliable goal detection
- Clear, easy-to-use game interface

Solutions / Learning

- Frequent tests and bug fixes
- Clear controls, instructions and colours

Phase-III Roadmap

1. Welcome, difficulty and custom-size setup
2. Custom wall editor with reachability check
3. Add A*, Dijkstra and Greedy BFS
4. Pause/resume/reset and live search
5. Compare algorithms; top-5 leaderboard

Finish testing, interface refinements and final report after implementing the remaining features.

Expected Outcomes & References

Expected Final Outcomes

- Interactive maze generator and solver
- Compare DFS, BFS and planned algorithms
- Visualize search paths and statistics
- Demonstrate DSA and OOP applications
- Custom maze creation and editing

References

1. Phase-II PBL Project Report
2. Project GitHub repository
3. Other research references not listed in report

Thank You

Questions & Suggestions

Potential applications discussed in the report include games, robot movement and navigation.